

Chapter 6. Collecting data with streams

This chapter covers

- Creating and using a collector with the `Collectors` class
- Reducing streams of data to a single value
- Summarization as a special case of reduction
- Grouping and partitioning data
- Developing your own custom collectors

You learned in the previous chapter that streams help you process collections with database-like operations. You can view Java 8 streams as fancy lazy iterators of sets of data. They support two types of operations: intermediate operations such as `filter` or `map` and terminal operations such as `count`, `findFirst`, `forEach`, and `reduce`. Intermediate operations can be chained to convert a stream into another stream. These operations don't consume from a stream; their purpose is to set up a pipeline of streams. By contrast, terminal operations *do* consume from a stream—to produce a final result (for example, returning the largest element in a stream). They can often shorten computations by optimizing the pipeline of a stream.

We already used the `collect` terminal operation on streams in [chapters 4](#) and [5](#), but we employed it there mainly to combine all the elements of a stream into a `List`. In this chapter, you'll discover that `collect` is a reduction operation, like `reduce`, that takes as an argument various recipes for accumulating the elements of a stream into a summary result. These recipes are defined by a new `Collector` interface, so it's important to distinguish `Collection`, `Collector`, and `collect`!

Here are some example queries of what you'll be able to do using `collect` and collectors:

- Group a list of transactions by currency to obtain the sum of the values of all transactions with that currency (returning a `Map<Currency,`

Other formats available



Audiobook



into two groups: expensive and not expensive, `List<Transaction>>)`

such as grouping transactions by cities

and then further categorizing by whether they're expensive or not (returning a `Map<String, Map<Boolean, List<Transaction>>>)`

Excited? Great. Let's start by exploring an example that benefits from collectors. Imagine a scenario where you have a `List of Transactions`, and you want to group them based on their nominal currency. Prior to Java 8, even a simple use case like this is cumbersome to implement, as shown in the following listing.

Listing 6.1. Grouping transactions by currency in imperative style

```
Map<Currency, List<Transaction>> transactionsByCurrencies =
    new HashMap<>();

for (Transaction transaction : transactions) {
    Currency currency = transaction.getCurrency();
    List<Transaction> transactionsForCurrency =
        transactionsByCurrencies.get(currency);
    if (transactionsForCurrency == null) {
        transactionsForCurrency = new ArrayList<>();
        transactionsByCurrencies
            .put(currency, transactionsForCurrency);
    }
    transactionsForCurrency.add(transaction);
}
```

- **1 Creates the Map where the grouped transaction will be accumulated**
- **2 Iterates the List of Transactions**
- **3 Extracts the Transaction's currency**
- **4 If there's no entry in the grouping Map for this currency, creates it**
- **5 Adds the currently traversed Transaction to the List of Transactions with the same currency**

If you're an experienced Java developer, you'll probably feel comfortable writing something like this, but you have to admit that it's a lot of code for such a simple task. Even worse, this is probably harder to read than to write! The purpose of the code isn't immediately evident at first glance, even though it can be expressed in a straightforward manner in plain English: "Group a list of transactions by their currency." As you'll learn in this chapter, you can achieve exactly the same result with a single statement by using a more general `Collector` parameter to the `collect` method on stream rather than the `toList` special case used in the previous chapter:

Other formats available



Audiobook



```
transaction>> transactionsByCurrencies =
    stream().collect(groupingBy(Transaction::getCurrency
```

The comparison is quite embarrassing, isn't it?

6.1. Collectors in a nutshell

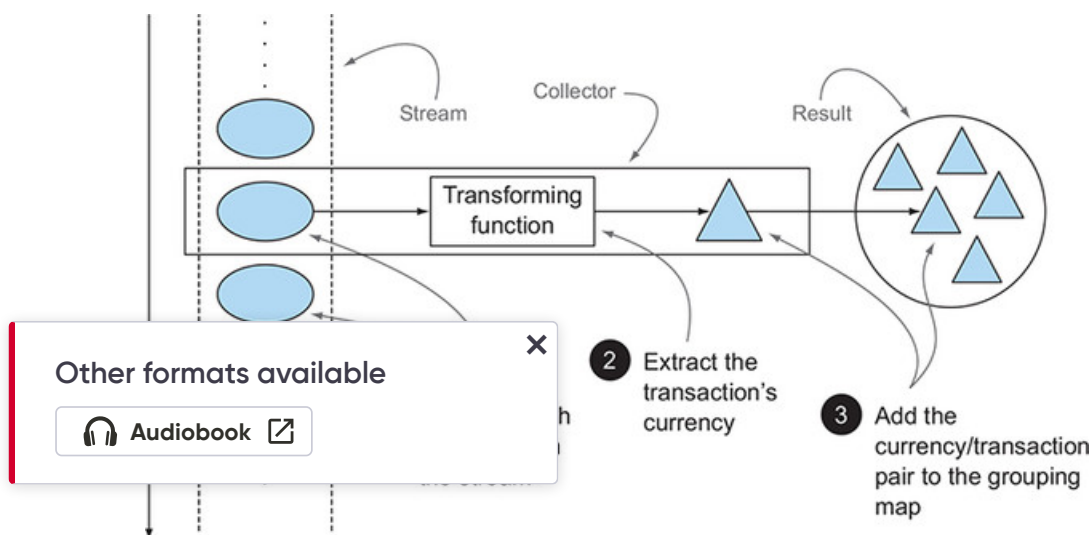
The previous example clearly shows one of the main advantages of functional-style programming over an imperative approach: you have to formulate the result you want to obtain the “what” and not the steps performed to obtain it, the “how.” In the previous example, the argument passed to the `collect` method is an implementation of the `Collector` interface, which is a recipe for how to build a summary of the elements in the stream. In the previous chapter, the `toList` recipe said, “Make a list of each element in turn.” In this example, the `groupingBy` recipe says, “Make a `Map` whose keys are (currency) buckets and whose values are a list of elements in those buckets.”

The difference between the imperative and functional versions of this example is even more pronounced if you perform multilevel groupings: in that case the imperative code quickly becomes harder to read, maintain, and modify due to the number of deeply nested loops and conditions required. In comparison, the functional-style version, as you’ll discover in [section 6.3](#), can be easily enhanced with an additional collector.

6.1.1. Collectors as advanced reductions

This last observation brings up another typical benefit of a well-designed functional API: its higher degree of composability and reusability. Collectors are extremely useful, because they provide a concise yet flexible way to define the criteria that `collect` uses to produce the resulting collection. More specifically, invoking the `collect` method on a stream triggers a reduction operation (parameterized by a `Collector`) on the elements of the stream itself. This *reduction operation*, illustrated in [figure 6.1](#), does for you internally what you had to code imperatively in [listing 6.1](#). It traverses each element of the stream and lets the `Collector` process them.

Figure 6.1. The reduction process grouping the transactions by currency



Typically, the `Collector` applies a transforming function to the element. Quite often this is the identity transformation, which has no effect (for example, as in `toList`). The function then accumulates the result in a data structure that forms the final output of this process. For instance, in our transaction-grouping example shown previously, the transformation function extracts the currency from each transaction, and subsequently the transaction itself is accumulated in the resulting `Map`, using the currency as key.

The implementation of the methods of the `Collector` interface defines how to perform a reduction operation on a stream, such as the one in our currency example. We'll investigate how to create customized collectors in [sections 6.5](#) and [6.6](#). But the `Collectors` utility class provides lots of static factory methods to conveniently create an instance of the most common collectors that are ready to use. The most straightforward and frequently used collector is the `toList` static method, which gathers all the elements of a stream into a `List`:

```
List<Transaction> transactions =  
    transactionStream.collect(Collectors.toList());
```

6.1.2. Predefined collectors

In the rest of this chapter, we'll mainly explore the features of the predefined collectors, those that can be created from the factory methods (such as `groupingBy`) provided by the `Collectors` class. These offer three main functionalities:

- Reducing and summarizing stream elements to a single value
- Grouping elements
- Partitioning elements

We start with collectors that allow you to reduce and summarize. These are handy in a variety of use cases, such as finding the total amount of the transacted values in the list of transactions in the previous example.

You'll then see how to group the elements of a stream, generalizing the previous example to multiple levels of grouping or combining different collectors to apply further reduction operations on each of the resulting subgroups. We'll also describe *partitioning* as a special case of grouping,

Other formats available



Audiobook



At the end of [section 6.4](#) you'll find a table summarizing all the predefined collectors explored in this chapter. Finally, in [section 6.5](#) you'll learn more about the `Collector` interface before you explore (in [section](#)

[6.6](#)) how you can create your own custom collectors to be used in the cases not covered by the factory methods of the `Collectors` class.

6.2. Reducing and summarizing

To illustrate the range of possible collector instances that can be created from the `Collectors` factory class, we'll reuse the domain we introduced in the previous chapter: a menu consisting of a list of delicious dishes!

As you learned, collectors (the parameters to the `stream` method `collect`) are typically used in cases where it's necessary to reorganize the stream's items into a collection. But more generally, they can be used every time you want to combine all the items in the stream into a single result. This result can be of any type, as complex as a multilevel map representing a tree or as simple as a single integer, perhaps representing the sum of all the calories in the menu. We'll look at both of these result types: single integers in [section 6.2.2](#) and multilevel grouping in [section 6.3.1](#).

As a first simple example, let's count the number of dishes in the menu, using the collector returned by the `counting` factory method:

```
long howManyDishes = menu.stream().collect(Collectors.counting());
```

You can write this far more directly as

```
long howManyDishes = menu.stream().count();
```

but the `counting` collector can be useful when used in combination with other collectors, as we'll demonstrate later.

In the rest of this chapter, we'll assume that you've imported all the static factory methods of the `Collectors` class with

```
import static java.util.stream.Collectors.*;
```

so you can write `counting()` instead of `Collectors.counting()` and so on.

Other formats available



Audiobook



predefined collectors by looking at how minimum values in a stream.

6.2.1. Finding maximum and minimum in a stream of values

Suppose you want to find the highest-calorie dish in the menu. You can use two collectors, `Collectors.maxBy` and `Collectors.minBy`, to calcu-

late the maximum or minimum value in a stream. These two collectors take a `Comparator` as argument to compare the elements in the stream. Here you create a `Comparator` comparing dishes based on their calorie content and pass it to `Collectors.maxBy`:

```
Comparator<Dish> dishCaloriesComparator =
    Comparator.comparingInt(Dish::getCalories);
Optional<Dish> mostCalorieDish =
    menu.stream()
        .collect(maxBy(dishCaloriesComparator));
```

You may wonder what the `Optional<Dish>` is about. To answer this we have to ask the question, "What if menu were empty?" There's no dish to return! Java 8 introduces `Optional`, which is a container that may or may not contain a value. Here it perfectly represents the idea that there may or may not be a dish returned. We briefly mentioned it in [chapter 5](#) when you encountered the method `findAny`. Don't worry about it for now; we devote [chapter 11](#) to the study of `Optional<T>` and its operations.

Another common reduction operation that returns a single value is to sum the values of a numeric field of the objects in a stream. Alternatively, you may want to average the values. Such operations are called *summarization* operations. Let's see how you can express them using collectors.

6.2.2. Summarization

The `Collectors` class provides a specific factory method for summing: `Collectors.summingInt`. It accepts a function that maps an object into the `int` that has to be summed and returns a collector that, when passed to the usual `collect` method, performs the requested summarization. For instance, you can find the total number of calories in your menu list with

```
int totalCalories = menu.stream().collect(summingInt(Dish::getCalories))
```

Here the collection process proceeds as illustrated in [figure 6.2](#). While traversing the stream each dish is mapped into its number of calories, and that number is added to an accumulator starting from an initial val-

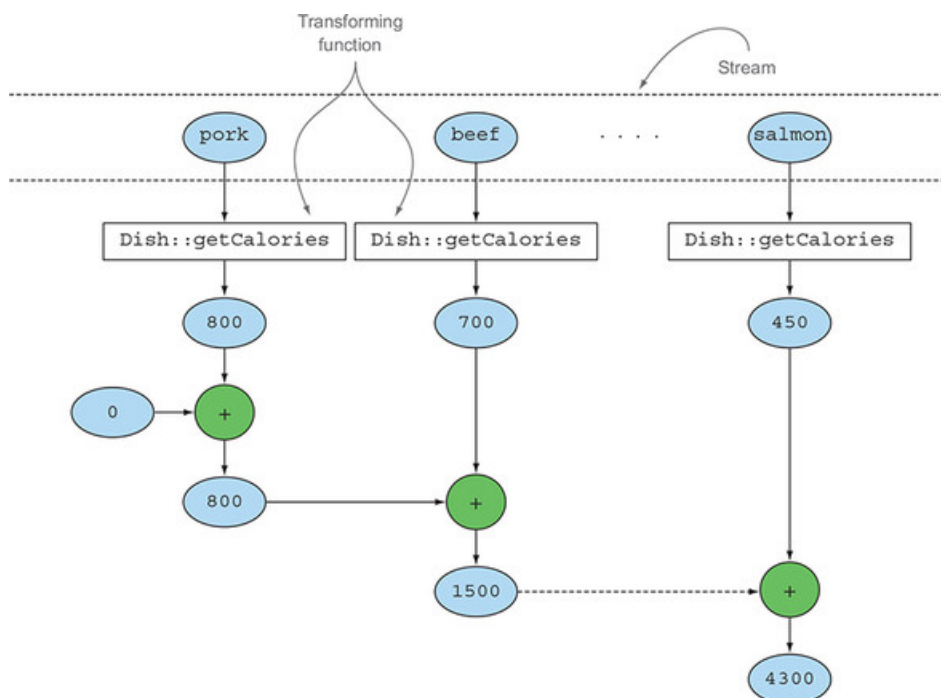
Other formats available



Audiobook



process of the `summingInt` collector



The `Collectors.summingLong` and `Collectors.summingDouble` methods behave exactly the same way and can be used where the field to be summed is respectively a long or a double.

But there's more to summarization than mere summing. A `Collectors.averagingInt`, together with its `averagingLong` and `averagingDouble` counterparts, is also available to calculate the average of the same set of numeric values:

```
double avgCalories =
    menu.stream().collect(averagingInt(Dish::getCalories));
```

So far, you've seen how to use collectors to count the elements in a stream, find the maximum and minimum values of a numeric property of those elements, and calculate their sum and average. Quite often, though, you may want to retrieve two or more of these results, and possibly you'd like to do it in a single operation. In this case, you can use the collector returned by the `summarizingInt` factory method. For example, you can count the elements in the menu and obtain the sum, average, maximum, and minimum of the calories contained in each dish with a single summarizing operation:

```
IntSummaryStatistics menuStatistics =
    menu.stream().collect(summarizingInt(Dish::getCalories));
```

Other formats available

 Audiobook 

information in a class called

`IntSummaryStatistics` that provides convenient getter methods to access the results. Printing the `menu-Statistic` object produces the following output:


```
IntSummaryStatistics{count=9, sum=4300, min=120,  
                    average=477.777778, max=800}
```

As usual, there are corresponding `summarizingLong` and `summarizingDouble` factory methods with associated types `LongSummaryStatistics` and `DoubleSummaryStatistics`. These are used when the property to be collected is a primitive-type `long` or a `double`.

6.2.3. Joining Strings

The collector returned by the `joining` factory method concatenates into a single string, all strings resulting from invoking the `toString` method on each object in the stream. This means you can concatenate the names of all the dishes in the menu as follows:

```
String shortMenu = menu.stream().map(Dish::getName).collect(joining());
```

Note that `joining` internally makes use of a `StringBuilder` to append the generated strings into one. Also note that if the `Dish` class had a `toString` method returning the dish's name, you'd obtain the same result without needing to map over the original stream with a function extracting the name from each dish:

```
String shortMenu = menu.stream().collect(joining());
```

Both produce the string

```
porkbeefchickenfrench friesricesseason fruitpizzaprawnssalmon
```

which is hard to read. Fortunately, the `joining` factory method is overloaded, with one of its overloaded variants taking a string used to delimit two consecutive elements, so you can obtain a comma-separated list of the dishes' names with

```
String shortMenu = menu.stream().map(Dish::getName).collect(joining(", "
```

Other formats available



Audiobook [↗](#)



```
french fries, rice, season fruit, pizza, prawns, se
```

Until now, we've explored various collectors that reduce a stream to a single value. In the next section, we'll demonstrate how all the reduction

processes of this form are special cases of the more general reduction collector provided by the `Collectors` `reducing` factory method.

6.2.4. Generalized summarization with reduction

All the collectors we've discussed so far are, in reality, only convenient specializations of a reduction process that can be defined using the `reducing` factory method. The `Collectors.reducing` factory method is a generalization of all of them. The special cases discussed earlier are arguably provided only for programmer convenience. (But remember that programmer convenience and readability are of prime importance!) For instance, it's *possible* to calculate the total calories in your menu with a collector created from the `reducing` method as follows:

```
int totalCalories = menu.stream().collect(reducing(
    0, Dish::getCalories, (i, j) -> i + j
```

It takes three arguments:

- The first argument is the starting value of the reduction operation and will also be the value returned in the case of a stream with no elements, so clearly `0` is the appropriate value in the case of a numeric sum.
- The second argument is the same function you used in [section 6.2.2](#) to transform a dish into an `int` representing its calorie content.
- The third argument is a `BinaryOperator` that aggregates two items into a single value of the same type. Here, it sums two `ints`.

Similarly, you could find the highest-calorie dish using the one-argument version of `reducing` as follows:

```
Optional<Dish> mostCalorieDish =
    menu.stream().collect(reducing(
        (d1, d2) -> d1.getCalories() > d2.getCalories() ? d1 : d2));
```

You can think of the collector created with the one-argument `reducing` factory method as a particular case of the three-argument method, which uses the first item in the stream as a starting point and an *identity function* (a function that returns its input argument as is) as a transformation

Other formats available



✕ The one-argument `reducing` collector is then passed to the `collect` method of an `Optional` object, as discussed in [section 6.2.1](#), for this reason it returns an `Optional<Dish>` object.

We've discussed reductions a lot in the previous chapter and this one. You may wonder what the differences between the `collect` and `reduce` methods of the stream interface are, because often you can obtain the same results using either method. For instance, you can achieve what is done by the `toList` Collector using the `reduce` method as follows:

```
Stream<Integer> stream = Arrays.asList(1, 2, 3, 4, 5, 6).stream();
List<Integer> numbers = stream.reduce(
    new ArrayList<Integer>(),
    (List<Integer> l, Integer e) -> {
        l.add(e);
        return l; },
    (List<Integer> l1, List<Integer> l2) -> {
        l1.addAll(l2);
        return l1; });
```

This solution has two problems: a semantic one and a practical one. The semantic problem lies in the fact that the `reduce` method is meant to combine two values and produce a new one; it's an immutable reduction. In contrast, the `collect` method is designed to mutate a container to accumulate the result it's supposed to produce. This means that the previous snippet of code is misusing the `reduce` method, because it's mutating in place the `List` used as accumulator. As you'll see in more detail in the next chapter, using the `reduce` method with the wrong semantic is also the cause of a practical problem: this reduction process can't work in parallel, because the concurrent modification of the same data structure operated by multiple threads can corrupt the `List` itself. In this case, if you want thread safety, you'll need to allocate a new `List` every time, which would impair performance by object allocation. This is the main reason why the `collect` method is useful for expressing reduction working on a mutable container but crucially in a parallel-friendly way, as you'll learn later in the chapter.

Collection framework flexibility: doing the same operation in different ways

You can further simplify the previous sum example using the reducing collector by using a reference to the `sum` method of the `Integer` class in-

Other formats available

 Audiobook 



... you used to encode the same operation.

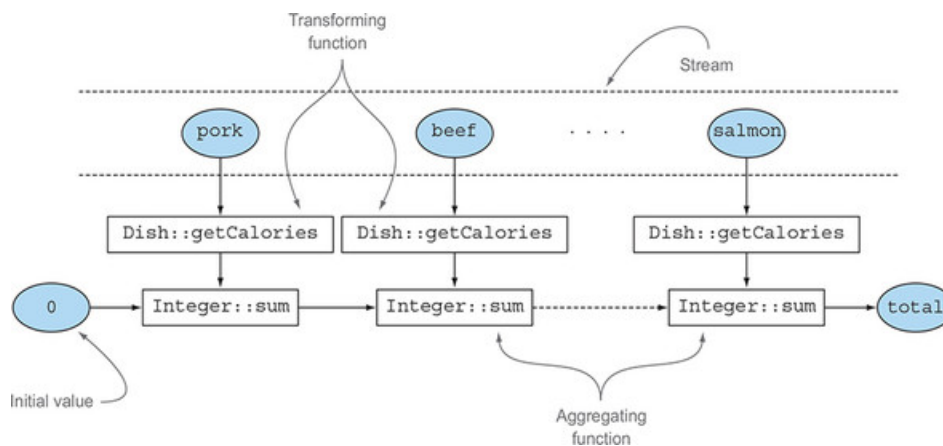
```
u.stream().collect(reducing(0,
    Dish::getCalories,
    Integer::sum));
```

1
2
3

- 1 Initial value
- 2 Transformation function
- 3 Aggregating function

Logically, this reduction operation proceeds as shown in [figure 6.3](#), where an accumulator—initialized with a starting value—is iteratively combined using an aggregating function, with the result of the application of the transforming function on each element of the stream.

Figure 6.3. The reduction process calculating the total number of calories in the menu



The counting collector we mentioned at the beginning of [section 6.2](#) is, in reality, similarly implemented using the three-argument reducing factory method. It transforms each element in the stream into an object of type Long with value 1 and then sums all these ones. It is implemented as follows:

```
public static <T> Collector<T, ?, Long> counting() {
    return reducing(0L, e -> 1L, Long::sum);
}
```

Use of the generic ? wildcard

In the code snippet just shown, you probably noticed the ? wildcard, used as the second generic type in the signature of the collector returned by the counting factory method. You should already be familiar with this notation, especially if you use the Java Collection Framework quite frequently. But here it means only that the type of the collector's accumula-

Other formats available

Audiobook

the accumulator itself can be of any type. the signature of the method as originally , but in the rest of the chapter, we avoid any wildcard notation to keep the discussion as simple as possible.

We already observed in [chapter 5](#) that there's another way to perform the same operation without using a collector—by mapping the stream of dishes into the number of calories of each dish and then reducing this resulting stream with the same method reference used in the previous version:

```
int totalCalories =  
    menu.stream().map(Dish::getCalories).reduce(Integer::sum).get();
```

Note that, like any one-argument reduce operation on a stream, the invocation `reduce(Integer::sum)` doesn't return an `int` but an `Optional<Integer>` to manage the case of a reduction operation over an empty stream in a null-safe way. Here you extract the value inside the `Optional` object using its `get` method. Note that in this case using the `get` method is safe only because you're sure that the stream of dishes isn't empty. In general, as you'll learn in [chapter 10](#), it's safer to unwrap the value eventually contained in an `Optional` using a method that also allows you to provide a default, such as `orElse` or `orElseGet`. Finally, and even more concisely, you can achieve the same result by mapping the stream to an `IntStream` and then invoking the `sum` method on it:

```
int totalCalories = menu.stream().mapToInt(Dish::getCalories).sum();
```

Choosing the best solution for your situation

Once again, this demonstrates how functional programming in general (and the new API based on functional-style principles added to the Collections framework in Java 8 in particular) often provides multiple ways to perform the same operation. This example also shows that collectors are somewhat more complex to use than the methods directly available on the Streams interface, but in exchange they offer higher levels of abstraction and generalization and are more reusable and customizable.

Our suggestion is to explore the largest number of solutions possible to the problem at hand, but always choose the most specialized one that's general enough to solve it. This is often the best decision for both readability and performance reasons. For instance, to calculate the total calories in our menu, we'd prefer the last solution (using `IntStream`) because it's the most concise and likely

Other formats available



Audiobook



also the most readable one. At the same time, `IntStream` seems best, because `IntStream` lets us avoid boxing, or implicit conversions from `Integer` to `int`.

Next, test your understanding of how reducing can be used as a generalization of other collectors by working through the exercise in quiz 6.1.

Quiz 6.1: Joining strings with reducing

Which of the following statements using the `reducing` collector are valid replacements for this joining collector (as used in [section 6.2.3](#))?

```
String shortMenu = menu.stream().map(Dish::getName).collect(joining());
```

1.

```
String shortMenu = menu.stream().map(Dish::getName)
    .collect( reducing( (s1, s2) -> s1 + s2 ) ).get()
```
2.

```
String shortMenu = menu.stream()
    .collect( reducing( (d1, d2) -> d1.getName() + d2.getName() )
    ).get();
```
3.

```
String shortMenu = menu.stream()
    .collect( reducing( "", Dish::getName, (s1, s2) -> s1 + s2 )
```

Answer:

Statements 1 and 3 are valid, whereas 2 doesn't compile.

1. This converts each dish in its name, as done by the original statement using the joining collector, and then reduces the resulting stream of strings using a `String` as accumulator and appending to it the names of the dishes one by one.
2. This doesn't compile because the one argument that `reducing` accepts is a `BinaryOperator<T>` that's a `BiFunction<T, T, T>`. This means that it wants a function taking two arguments and returns a value of the same type, but the lambda expression used there has two dishes as arguments but returns a string.
3. This starts the reduction process with an empty string as the accumulator, and when traversing the stream of dishes, it converts each dish to its name and appends this name to the accumulator. Note that, as we mentioned, `reducing` doesn't need the three arguments to return an `Optional` because in the case of an empty stream it can return a more meaningful value, which is the empty string used as the initial

Other formats available



Statements 1 and 3 are valid replacements for the joining collector used here to demonstrate how the re-

ducing one can be seen, at least conceptually, as a generalization of all other collectors discussed in this chapter. Nevertheless, for all practical purposes we always suggest using the joining collector for both readability and performance reasons.

6.3. Grouping

A common database operation is to group items in a set, based on one or more properties. As you saw in the earlier transactions-currency-grouping example, this operation can be cumbersome, verbose, and error-prone when implemented with an imperative style. But it can be easily translated in a single, readable statement by rewriting it in a more functional style as encouraged by Java 8. As a second example of how this feature works, suppose you want to classify the dishes in the menu according to their type, putting the ones containing meat in a group, the ones with fish in another group, and all others in a third group. You can easily perform this task using a collector returned by the `Collectors.groupingBy` factory method, as follows:

```
Map<Dish.Type, List<Dish>> dishesByType =  
    menu.stream().collect(groupingBy(Dish::getType));
```

This will result in the following Map:

```
{FISH=[prawns, salmon], OTHER=[french fries, rice, season fruit, pizza],  
  MEAT=[pork, beef, chicken]}
```

Here, you pass to the `groupingBy` method a Function (expressed in the form of a method reference) extracting the corresponding `Dish.Type` for each `Dish` in the stream. We call this Function a *classification* function specifically because it's used to classify the elements of the stream into different groups. The result of this grouping operation, shown in [figure 6.4](#), is a Map having as map key the value returned by the classification function and as corresponding map value a list of all the items in the stream having that classified value. In the menu-classification example, a key is the type of dish, and its value is a list containing all the dishes of that type.

Figure 6.4. Classification of an item in the stream during the grouping process



But it isn't always possible to use a method reference as a classification function, because you may wish to classify using something more complex than a simple property accessor. For instance, you could decide to classify as "diet" all dishes with 400 calories or fewer, set to "normal" the dishes having between 400 and 700 calories, and set to "fat" the ones with more than 700 calories. Because the author of the Dish class unhelpfully didn't provide such an operation as a method, you can't use a method reference in this case, but you can express this logic in a lambda expression:

```
public enum CaloricLevel { DIET, NORMAL, FAT }
Map<CaloricLevel, List<Dish>> dishesByCaloricLevel = menu.stream().collect(
    groupingBy(dish -> {
        if (dish.getCalories() <= 400) return CaloricLevel.DIET;
        else if (dish.getCalories() <= 700) return CaloricLevel.NORMAL;
        else return CaloricLevel.FAT;
    } ));
```

Now you've seen how to group the dishes in the menu, both by their type and by calories, but it could be also quite common that you may need to further manipulate the results of the original grouping, and in the next section we'll show how you achieve this.

6.3.1. Manipulating grouped elements

Frequently after performing a grouping operation you may need to manipulate the elements in each resulting group. Suppose, for example, that you want to filter only the caloric dishes, let's say the ones with more than 500 calories. You may argue that in this case you could apply this filtering predicate before the grouping like the following:

```
Map<Dish.Type, List<Dish>> caloricDishesByType =
    menu.stream().filter(dish -> dish.getCalories() > 500)
        .collect(groupingBy(Dish::getType));
```

This solution works but has a possibly relevant drawback. If you try to use it on the dishes in our menu, you will obtain a Map like the following:

```
{OTHER=[french fries, pizza], MEAT=[pork, beef]}
```

Other formats available



✗ Because there is no dish of type FISH satisfying the key totally disappeared from the result. To solve this problem the Collectors class overloads the groupingBy factory method, with one variant also taking a second argument of type Collector along with the usual classification function. In this way, it's possible to move the filtering predicate inside this second Collector, as follows:


```
Map<Dish.Type, List<Dish>> caloricDishesByType =
    menu.stream()
        .collect(groupingBy(Dish::getType,
            filtering(dish -> dish.getCalories() > 500, toList()))
```

The filtering method is another static factory method of the Collectors class accepting a Predicate to filter the elements in each group and a further Collector that is used to regroup the filtered elements. In this way, the resulting Map will also keep an entry for the FISH type even if it maps an empty List:

```
{OTHER=[french fries, pizza], MEAT=[pork, beef], FISH=[]}
```

Another even more common way in which it could be useful to manipulate the grouped elements is transforming them through a mapping function. To this purpose, similarly to what you have seen for the filtering Collector, the Collectors class provides another Collector through the mapping method that accepts a mapping function and another Collector used to gather the elements resulting from the application of that function to each of them. By using it you can, for instance, convert each Dish in the groups into their respective names in this way:

```
Map<Dish.Type, List<String>> dishNamesByType =
    menu.stream()
        .collect(groupingBy(Dish::getType,
            mapping(Dish::getName, toList())));
```

Note that in this case each group in the resulting Map is a List of Strings rather than one of Dishes as it was in the former examples. You could also use a third Collector in combination with the groupingBy to perform a flatMap transformation instead of a plain map. To demonstrate how this works let's suppose that we have a Map associating to each Dish a list of tags as it follows:

```
Map<String, List<String>> dishTags = new HashMap<>();
dishTags.put("pork", asList("greasy", "salty"));
dishTags.put("beef", asList("salty", "roasted"));
dishTags.put("chicken", asList("fried", "crisp"));
dishTags.put("french fries", asList("greasy", "fried"));
dishTags.put("pizza", asList("light", "natural"));
dishTags.put("salmon", asList("fresh", "natural"));
dishTags.put("prawns", asList("tasty", "salty"));
dishTags.put("chicken", asList("tasty", "salty"));
dishTags.put("prawns", asList("tasty", "roasted"));
dishTags.put("salmon", asList("delicious", "fresh"));
```

Other formats available



In case you are required to extract these tags for each group of type of dishes you can easily achieve this using the `flatMap` Collector:

```
Map<Dish.Type, Set<String>> dishNamesByType =
    menu.stream()
        .collect(groupingBy(Dish::getType,
            flatMapping(dish -> dishTags.get( dish.getName() ).stream
                toSet())));
```

Here for each Dish we are obtaining a List of tags. So analogously to what we have already seen in the former chapter, we need to perform a `flatMap` in order to flatten the resulting two-level list into a single one. Also note that this time we collected the result of the `flatMap` operations executed in each group into a Set instead of using a List as we did before, in order to avoid repetitions of same tags associated to more than one Dish in the same type. The Map resulting from this operation is then the following:

```
{MEAT=[salty, greasy, roasted, fried, crisp], FISH=[roasted, tasty, fresh, delicious], OTHER=[salty, greasy, natural, light, tasty, fresh, fried]}
```

Until this point we only used a single criterion to group the dishes in the menu, for instance by their type or by calories, but what if you want to use more than one criterion at the same time? Grouping is powerful because it composes effectively. Let's see how to do this.

6.3.2. Multilevel grouping

The two arguments `Collectors.groupingBy` factory method that we used in a former section to manipulate the elements in the groups resulting from the grouping operation can be used also to perform a two-level grouping. To achieve this you can pass to it a second inner `groupingBy` to the outer `groupingBy`, defining a second-level criterion to classify the stream's items, as shown in the next listing.

Listing 6.2. Multilevel grouping

```
Map<Dish.Type, Map<CaloricLevel, List<Dish>>> dishesByTypeCaloricLevel =
    menu.stream().collect(
        groupingBy(Dish::getType,
            groupingBy(dish -> {
                if (dish.getCalories() <= 400) return CaloricLevel.DIET;
                else if (dish.getCalories() <= 700) return CaloricLevel.NORMAL;
                else return CaloricLevel.FAT;
            })
        )
    );
```

Other formats available



- 1 First-level classification function
- 2 Second-level classification function

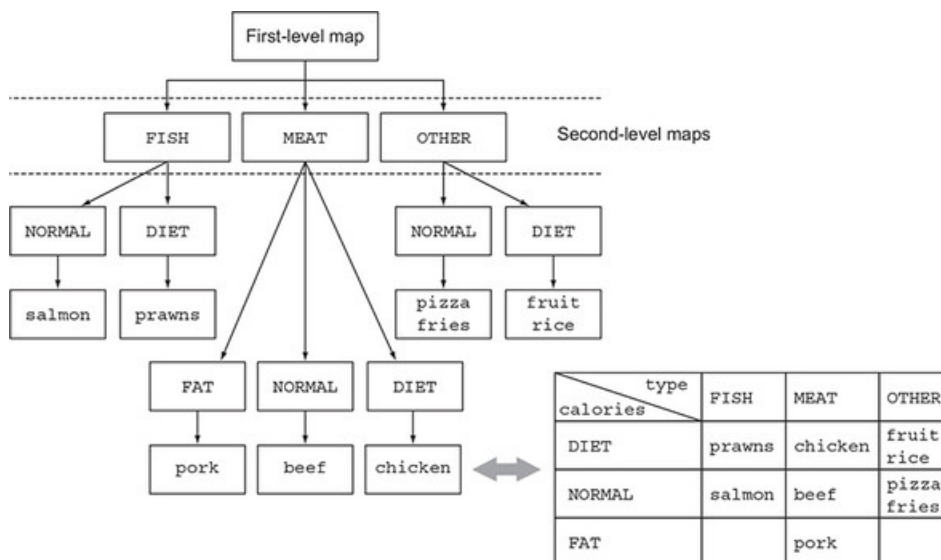
The result of this two-level grouping is a two-level Map like the following:

```
{MEAT={DIET=[chicken], NORMAL=[beef], FAT=[pork]},
  FISH={DIET=[prawns], NORMAL=[salmon]},
  OTHER={DIET=[rice, seasonal fruit], NORMAL=[french fries, pizza]}}
```

Here the outer Map has as keys the values generated by the first-level classification function: fish, meat, other. The values of this Map are in turn other Maps, having as keys the values generated by the second-level classification function: normal, diet, or fat. Finally, the second-level Maps have as values the List of the elements in the stream returning the corresponding first- and second-level key values when applied respectively to the first and second classification functions: salmon, pizza, and so on. This multilevel grouping operation can be extended to any number of levels, and an n -level grouping has as a result an n -level Map, modeling an n -level tree structure.

Figure 6.5 shows how this structure is also equivalent to an n -dimensional table, highlighting the classification purpose of the grouping operation.

Figure 6.5. Equivalence between n -level nested map and n -dimensional classification table



In general, it helps to think that `groupBy` works in terms of “buckets.”

Other formats available



Audiobook



✕ bucket for each key. You then collect the
the downstream collector and so on to

6.3.3. Collecting data in subgroups

In the previous section, you saw that it's possible to pass a second `groupBy` collector to the outer one to achieve a multilevel grouping. But more generally, the second collector passed to the first `groupBy` can be any type of collector, not just another `groupBy`. For instance, it's possible to count the number of Dishes in the menu for each type, by passing the counting collector as a second argument to the `groupBy` collector:

```
Map<Dish.Type, Long> typesCount = menu.stream().collect(
    groupingBy(Dish::getType, counting()));
```

The result is the following Map:

```
{MEAT=3, FISH=2, OTHER=4}
```

Also note that the regular one-argument `groupBy(f)`, where `f` is the classification function is, in reality, shorthand for `groupBy(f, toList())`.

To give another example, you could rework the collector you already used to find the highest-calorie dish in the menu to achieve a similar result, but now classified by the *type* of dish:

```
Map<Dish.Type, Optional<Dish>> mostCaloricByType =
    menu.stream()
        .collect(groupingBy(Dish::getType,
            maxBy(comparingInt(Dish::getCalories))));
```

The result of this grouping is then clearly a Map, having as keys the available types of Dishes and as values the `Optional<Dish>`, wrapping the corresponding highest-calorie Dish for a given type:

```
{FISH=Optional[salmon], OTHER=Optional[pizza], MEAT=Optional[pork]}
```

Note

Other formats available



Optional because this is the resulting type of `maxBy` factory method, but in reality if a given type, that type won't have an

`Optional.empty()` as value; it won't be present at all as a key in the Map. The `groupBy` collector lazily adds a new key in the grouping Map only the first time it finds an element in the stream, producing that key when applying on it the grouping criteria being used. This means that in this

case, the `Optional` wrapper isn't useful, because it's not modeling a value that could be possibly absent but is there incidentally, only because this is the type returned by the reducing collector.

Adapting the collector result to a different type

Because the `Optionals` wrapping all the values in the `Map` resulting from the last grouping operation aren't useful in this case, you may want to get rid of them. To achieve this, or more generally, to adapt the result returned by a collector to a different type, you could use the collector returned by the `Collectors.collectingAndThen` factory method, as shown in the following listing.

Listing 6.3. Finding the highest-calorie dish in each subgroup

```
Map<Dish.Type, Dish> mostCaloricByType =  
    menu.stream()  
        .collect(groupingBy(Dish::getType, 1  
                           collectingAndThen(  
                               maxBy(comparingInt(Dish::getCalories)), 2  
                               Optional::get)), 3
```

- **1 Classification function**
- **2 Wrapped collector**
- **3 Transformation function**

This factory method takes two arguments—the collector to be adapted and a transformation function—and returns another collector. This additional collector acts as a wrapper for the old one and maps the value it returns using the transformation function as the last step of the `collect` operation. In this case, the wrapped collector is the one created with `maxBy`, and the transformation function, `Optional::get`, extracts the value contained in the `Optional` returned. As we've said, here this is safe because the reducing collector will never return an `Optional.empty()`. The result is the following `Map`:

```
{FISH=salmon, OTHER=pizza, MEAT=pork}
```

Other formats available



note the following:

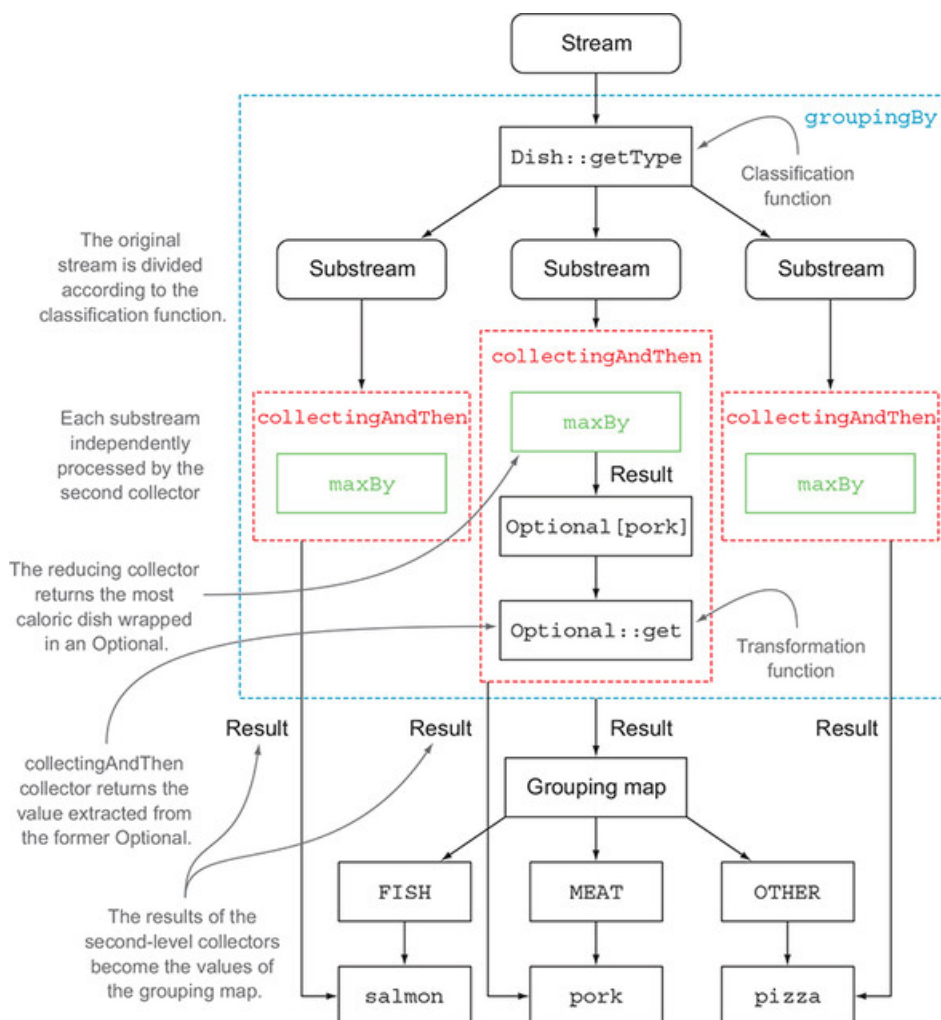
- The collectors are represented by the dashed lines, so `groupingBy` is the outermost one and groups the menu stream into three substreams

the nested collectors, and at first the way obvious. [Figure 6.6](#) helps you visualize the outermost layer and moving inward,

according to the different dishes' types.

- The `groupBy` collector wraps the `collectingAndThen` collector, so each substream resulting from the grouping operation is further reduced by this second collector.
- The `collectingAndThen` collector wraps in turn a third collector, the `maxBy` one.
- The reduction operation on the substreams is then performed by the reducing collector, but the `collectingAndThen` collector containing it applies the `Optional::get` transformation function to its result.
- The three transformed values, being the highest-calorie Dishes for a given type (resulting from the execution of this process on each of the three substreams), will be the values associated with the respective classification keys, the types of Dishes, in the `Map` returned by the `groupBy` collector.

Figure 6.6. Combining the effect of multiple collectors by nesting one inside the other



Other formats available



Audiobook



used in conjunction with `groupBy`

used as second argument to the `group-`

`ingBy` factory method will be used to perform a further reduction operation on all the elements in the stream classified into the same group. For example, you could also reuse the collector created to sum the calories of

all the dishes in the menu to obtain a similar result, but this time for each group of Dishes:

```
Map<Dish.Type, Integer> totalCaloriesByType =  
    menu.stream().collect(groupingBy(Dish::getType,  
                                     summingInt(Dish::getCalories)));
```

Yet another collector, commonly used in conjunction with `groupingBy`, is one generated by the mapping method. This method takes two arguments: a function transforming the elements in a stream and a further collector accumulating the objects resulting from this transformation. Its purpose is to adapt a collector accepting elements of a given type to one working on objects of a different type, by applying a mapping function to each input element before accumulating them. To see a practical example of using this collector, suppose you want to know which `CaloricLevels` are available in the menu for each type of `Dish`. You could achieve this result combining a `groupingBy` and a mapping collector, as follows:

```
Map<Dish.Type, Set<CaloricLevel>> caloricLevelsByType =  
    menu.stream().collect(  
        groupingBy(Dish::getType, mapping(dish -> {  
            if (dish.getCalories() <= 400) return CaloricLevel.DIET;  
            else if (dish.getCalories() <= 700) return CaloricLevel.NORMAL;  
            else return CaloricLevel.FAT; },  
        toSet() ));
```

Here the transformation function passed to the mapping method maps a `Dish` into its `CaloricLevel`, as you've seen before. The resulting stream of `CaloricLevels` is then passed to a `toSet` collector, analogous to the `toList` one, but accumulating the elements of a stream into a `Set` instead of into a `List`, to keep only the distinct values. As in earlier examples, this mapping collector will then be used to collect the elements in each substream generated by the grouping function, allowing you to obtain as a result the following `Map`:

```
{OTHER=[DIET, NORMAL], MEAT=[DIET, NORMAL, FAT], FISH=[DIET, NORMAL]}
```

From this you can easily figure out your choices. If you're in the mood for

Other formats available

 Audiobook 



could easily find a dish; likewise, if you're in the mood for a dish with lots of calories, you could satisfy your craving by choosing something from the meat section of the menu. For example, there are no guarantees about

what type of `Set` is returned. But by using `toCollection`, you can have more control. For example, you can ask for a `HashSet` by passing a constructor reference to it:


```
Map<Dish.Type, Set<CaloricLevel>> caloricLevelsByType =
menu.stream().collect(
    groupingBy(Dish::getType, mapping(dish -> {
        if (dish.getCalories() <= 400) return CaloricLevel.DIET;
        else if (dish.getCalories() <= 700) return CaloricLevel.NORMAL;
        else return CaloricLevel.FAT; },
        toCollection(HashSet::new) ));
```

6.4. Partitioning

Partitioning is a special case of grouping: having a predicate called a *partitioning function* as a classification function. The fact that the partitioning function returns a boolean means the resulting grouping Map will have a Boolean as a key type, and therefore, there can be at most two different groups—one for true and one for false. For instance, if you're vegetarian or have invited a vegetarian friend to have dinner with you, you may be interested in partitioning the menu into vegetarian and nonvegetarian dishes:

```
Map<Boolean, List<Dish>> partitionedMenu =
    menu.stream().collect(partitioningBy(Dish::isVegetarian));
```

- **1 Partitioning function**

This will return the following Map:

```
{false=[pork, beef, chicken, prawns, salmon],
 true=[french fries, rice, season fruit, pizza]}
```

So you could retrieve all the vegetarian dishes by getting from this Map the value indexed with the key true:

```
List<Dish> vegetarianDishes = partitionedMenu.get(true);
```

Note that you could achieve the same result by filtering the stream created from the menu List with the same predicate used for partitioning and then collecting the result in an additional List:

Other formats available



Audiobook



```
shes =
am().filter(Dish::isVegetarian).collect(toList())
```

6.4.1. Advantages of partitioning

Partitioning has the advantage of keeping both lists of the stream elements, for which the application of the partitioning function returns true

or false. In the previous example, you can obtain the List of the non-vegetarian Dishes by accessing the value of the key false in the partitionedMenu Map, using two separate filtering operations: one with the predicate and one with its negation. Also, as you already saw for grouping, the partitioningBy factory method has an overloaded version to which you can pass a second collector, as shown here:

```
Map<Boolean, Map<Dish.Type, List<Dish>>> vegetarianDishesByType =  
menu.stream().collect(  
    partitioningBy(Dish::isVegetarian, 1  
        groupingBy(Dish::getType)); 2
```

- **1 Partitioning function**
- **2 Second collector**

This will produce a two-level Map:

```
{false={FISH=[prawns, salmon], MEAT=[pork, beef, chicken]},  
true={OTHER=[french fries, rice, season fruit, pizza]}}
```

Here the grouping of the dishes by their type is applied individually to both of the substreams of vegetarian and nonvegetarian dishes resulting from the partitioning, producing a two-level Map that's similar to the one you obtained when you performed the two-level grouping in [section 6.3.1](#). As another example, you can reuse your earlier code to find the most caloric dish among both vegetarian and nonvegetarian dishes:

```
Map<Boolean, Dish> mostCaloricPartitionedByVegetarian =  
menu.stream().collect(  
    partitioningBy(Dish::isVegetarian,  
        collectingAndThen(maxBy(comparingInt(Dish::getCalories)),  
            Optional::get));
```

That will produce the following result:

```
{false=pork, true=pizza}
```

We started this section by saying that you can think of partitioning as a

Other formats available



With also noting that the Map implementation By is more compact and efficient as it partitions by true and false. In fact, the internal implementation is a specialized Map with two fields. The analogies between the groupingBy and partitioningBy collectors don't end here; as you'll see in the next quiz, you can also perform multilevel partitioning in a way similar to what you did for grouping in [section 6.3.1](#).

Quiz 6.2: Using partitioningBy

As you've seen, like the `groupingBy` collector, the `partitioningBy` collector can be used in combination with other collectors. In particular it could be used with a second `partitioningBy` collector to achieve a multilevel partitioning. What will be the result of the following multilevel partitionings?

1. `menu.stream().collect(partitioningBy(Dish::isVegetarian, partitioningBy(d -> d.getCalories() > 500)))`
2. `menu.stream().collect(partitioningBy(Dish::isVegetarian, partitioningBy(Dish::getType())));`
3. `menu.stream().collect(partitioningBy(Dish::isVegetarian, counting()));`

Answer:

1. This is a valid multilevel partitioning, producing the following two-level Map:

```
{ false={false=[chicken, prawns, salmon], true=[pork, beef]},  
  true={false=[rice, season fruit], true=[french fries, pizza]
```

2. This won't compile because `partitioningBy` requires a predicate, a function returning a boolean. And the method reference `Dish::getType` can't be used as a predicate.
3. This counts the number of items in each partition, resulting in the following Map:

```
{false=5, true=4}
```

To give one last example of how you can use the `partitioningBy` collec-

Other formats available



Audiobook



a model and look at something a bit more
g: partitioning numbers into prime and

6.4.2. Partitioning numbers into prime and nonprime

Suppose you want to write a method accepting as argument an `int n` and partitioning the first n natural numbers into prime and nonprime. But first, it will be useful to develop a predicate that tests to see if a given candidate number is prime or not:

```
public boolean isPrime(int candidate) {  
    return IntStream.range(2, candidate)           1  
                   .noneMatch(i -> candidate % i == 0); 2  
}
```

- **1 Generates a range of natural numbers starting from and including 2, up to but excluding candidate**
- **2 Returns true if the candidate isn't divisible for any of the numbers in the stream**

A simple optimization is to test only for factors less than or equal to the square root of the candidate:

```
public boolean isPrime(int candidate) {  
    int candidateRoot = (int) Math.sqrt((double) candidate);  
    return IntStream.rangeClosed(2, candidateRoot)  
                   .noneMatch(i -> candidate % i == 0);  
}
```

Now the biggest part of the job is done. To partition the first n numbers into prime and nonprime, it's enough to create a stream containing those n numbers, and reduce it with a `partitioningBy` collector using as predicate the `isPrime` method you just developed:

```
public Map<Boolean, List<Integer>> partitionPrimes(int n) {  
    return IntStream.rangeClosed(2, n).boxed()  
                   .collect(  
                       partitioningBy(candidate -> isPrime(candidate)));  
}
```

We've now covered all the collectors that can be created using the static factory methods of the `Collectors` class, showing practical examples of how they work. [Table 6.1](#) brings them all together with the type they return when applied to a `Stream<T>`, and a practical example of their use

Other formats available



✕ uStream.

ry methods of the `Collectors` class

Factory method	Returned type	Used to
toList	List<T>	Gather all the stream's items in a List. Example use: List<Dish> dishes = menuStream.collect(toList());
toSet	Set<T>	Gather all the stream's items in a Set, eliminating duplicates. Example use: Set<Dish> dishes = menuStream.collect(toSet());
toCollection	Collection<T>	Gather all the stream's items in the collection created by the provided supplier. Example use: Collection<Dish> dishes = menuStream.collect(toCollection(), ArrayList::new);
counting	Long	Count the number of items in the stream. Example use: long howManyDishes = menuStream.collect(counting());
summingInt	Integer	Sum the values of an Integer property of the items in the stream. Example use: int totalCalories = menuStream.collect(summingInt(Dish::getCalories));
averagingInt	Double	Calculate the average value of an Integer property of the items in the stream. Example use: double averageCalories = menuStream.collect(averagingInt(Dish::getCalories));
summarizingInt	IntSummaryStatistics	Collect statistics regarding an Integer

Other formats available



Audiobook



Factory method	Returned type	Used to
		property of the items in the stream, such as the maximum, minimum, total, and average. Example use: <code>IntSummaryStatistics menuStatistics = menuStream.collect(summarizingInt(Dish::getCalories));</code>
joining	String	Concatenate the strings resulting from the invocation of the <code>toString</code> method on each item of the stream. Example use: <code>String shortMenu = menuStream.map(Dish::getName).collect(joining(", "));</code>
maxBy	Optional<T>	An Optional wrapping the maximal element in this stream according to the given comparator or <code>Optional.empty()</code> if the stream is empty. Example use: <code>Optional<Dish> fattest = menuStream.collect(maxBy(comparingInt(Dish::getCalories)));</code>
minBy	Optional<T>	An Optional wrapping the minimal element in this stream according to the given comparator or <code>Optional.empty()</code> if the stream is empty.
reducing	The type produced by the reduction operation	Reduce the stream to a single value starting from an initial value used as accumulator

Other formats available



Audiobook



lightest =

`comparingInt(Dish::getCalories)));`

Factory method	Returned type	Used to
		and iteratively combining it with each item of the stream using a <code>BinaryOperator</code> .
Example use: <code>int totalCalories = menuStream.collect(reducing(0, Dish::getCalories, Integer::sum));</code>		
<code>collectingAndThen</code>	The type returned by the transforming function	Wrap another collector and apply a transformation function to its result.
Example use: <code>int howManyDishes = menuStream.collect(collectingAndThen(toList(), List::size));</code>		
<code>groupingBy</code>	<code>Map<K, List<T>></code>	Group the items in the stream based on the value of one of their properties and use those values as keys in the resulting <code>Map</code> .
Example use: <code>Map<Dish.Type, List<Dish>> dishesByType = menuStream.collect(groupingBy(Dish::getType));</code>		
<code>partitioningBy</code>	<code>Map<Boolean, List<T>></code>	Partition the items in the stream based on the result of the application of a predicate to each of them.
Example use: <code>Map<Boolean, List<Dish>> vegetarianDishes = menuStream.collect(partitioningBy(Dish::isVegetarian));</code>		

Other formats available



Audiobook



ing of the chapter, all these collectors im-
e, so in the remaining part of the chapter
more detail. We investigate the methods
in that interface and then explore how you can implement your own
collectors.

6.5. The Collector interface

The `Collector` interface consists of a set of methods that provide a blueprint for how to implement specific reduction operations (collectors). You've seen many collectors that implement the `Collector` interface, such as `toList` or `groupingBy`. This also implies that you're free to create customized reduction operations by providing your own implementation of the `Collector` interface. In [section 6.6](#) we'll show how you can implement the `Collector` interface to create a collector to partition a stream of numbers into prime and nonprime more efficiently than what you've seen so far.

To get started with the `Collector` interface, we focus on one of the first collectors you encountered at the beginning of this chapter: the `toList` factory method, which gathers all the elements of a stream in a `List`. We said that you'll frequently use this collector in your day-to-day job, but it's also one that, at least conceptually, is straightforward to develop. Investigating in more detail how this collector is implemented is a good way to understand how the `Collector` interface is defined and how the functions returned by its methods are internally used by the `collect` method.

Let's start by taking a look at the definition of the `Collector` interface in the next listing, which shows the interface signature together with the five methods it declares.

Listing 6.4. The `Collector` interface

```
public interface Collector<T, A, R> {
    Supplier<A> supplier();
    BiConsumer<A, T> accumulator();
    Function<A, R> finisher();
    BinaryOperator<A> combiner();
    Set<Characteristics> characteristics();
}
```

In this listing, the following definitions apply:

- `T` is the generic type of the items in the stream to be collected.
- `A` is the type of the accumulator, the object on which the partial result will be accumulated during the collection process.
- `R` is the type of the object (typically, but not always, the collection) re-

Other formats available



tion.

ent a `ToListCollector<T>` class that gathers all the elements of a `Stream<T>` into a `List<T>` having the following signature

```
public class ToListCollector<T> implements Collector<T, List<T>, List<T>
```

where, as we'll clarify shortly, the object used for the accumulation process will also be the final result of the collection process.

6.5.1. Making sense of the methods declared by Collector interface

We can now analyze the five methods declared by the `Collector` interface one by one. When we do so, you'll notice that each of the first four methods returns a function that will be invoked by the `collect` method, whereas the fifth one, `characteristics`, provides a set of characteristics that's a list of hints used by the `collect` method itself to know which optimizations (for example, parallelization) it's allowed to employ while performing the reduction operation.

Making a new result container: the supplier method

The `supplier` method has to return a `Supplier` of an empty accumulator—a parameterless function that when invoked creates an instance of an empty accumulator used during the collection process. Clearly, for a collector returning the accumulator itself as result, like our `ToListCollector`, this empty accumulator will also represent the result of the collection process when performed on an empty stream. In our `ToListCollector` the `supplier` will then return an empty `List`, as follows:

```
public Supplier<List<T>> supplier() {  
    return () -> new ArrayList<T>();  
}
```

Note that you could also pass a constructor reference:

```
public Supplier<List<T>> supplier() {  
    return ArrayList::new;  
}
```

Adding an element to a result container: the accumulator method

The `accumulator` method returns the function that performs the reduction operation. When traversing the n th element in the stream, this function is applied with two arguments, the accumulator being the result of the reduction (after having collected the first $n-1$ items of the stream) and

Other formats available



on returns void because the accumulator that its internal state is changed by the effect of the traversed element. For `ToListCollector`, this function merely has to add the current item to the list containing the already traversed ones:

```
public BiConsumer<List<T>, T> accumulator() {  
    return (list, item) -> list.add(item);  
}
```

You could instead use a method reference, which is more concise:

```
public BiConsumer<List<T>, T> accumulator() {  
    return List::add;  
}
```

Applying the final transformation to the result container: the finisher method

The finisher method has to return a function that's invoked at the end of the accumulation process, after having completely traversed the stream, in order to transform the accumulator object into the final result of the whole collection operation. Often, as in the case of the `ToListCollector`, the accumulator object already coincides with the final expected result. As a consequence, there's no need to perform a transformation, so the finisher method has to return the identity function:

```
public Function<List<T>, List<T>> finisher() {  
    return Function.identity();  
}
```

These first three methods are enough to execute a sequential reduction of the stream that, at least from a logical point of view, could proceed as in [figure 6.7](#). The implementation details are a bit more difficult in practice due to both the lazy nature of the stream, which could require a pipeline of other intermediate operations to execute before the collect operation, and the possibility, in theory, of performing the reduction in parallel.

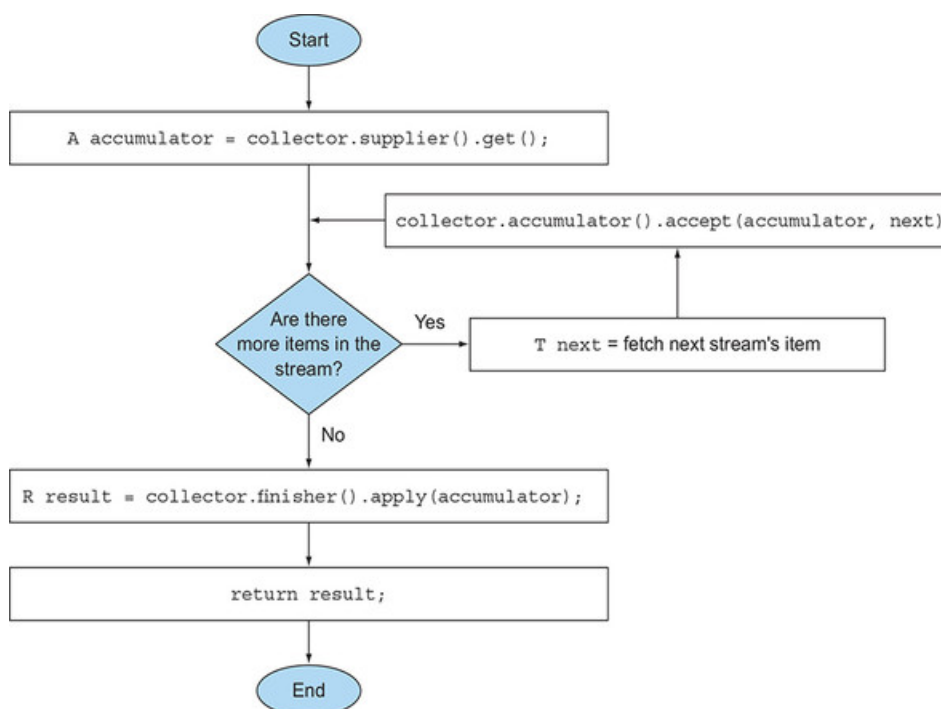
Figure 6.7. Logical steps of the sequential reduction process

Other formats available



Audiobook





Merging two result containers: the combiner method

The combiner method, the last of the four methods that return a function used by the reduction operation, defines how the accumulators resulting from the reduction of different subparts of the stream are combined when the subparts are processed in parallel. In the `toList` case, the implementation of this method is simple; add the list containing the items gathered from the second subpart of the stream to the end of the list obtained when traversing the first subpart:

```
public BinaryOperator<List<T>> combiner() {
    return (list1, list2) -> {
        list1.addAll(list2);
        return list1; }
}
```

The addition of this fourth method allows a parallel reduction of the stream. This uses the fork/join framework introduced in Java 7 and the `Splitter` abstraction that you'll learn about in the next chapter. It follows a process similar to the one shown in [figure 6.8](#) and described in detail here.

- The original *stream* is recursively split in substreams until a condition defining whether a stream needs to be further divided becomes false

Other formats available



Audiobook

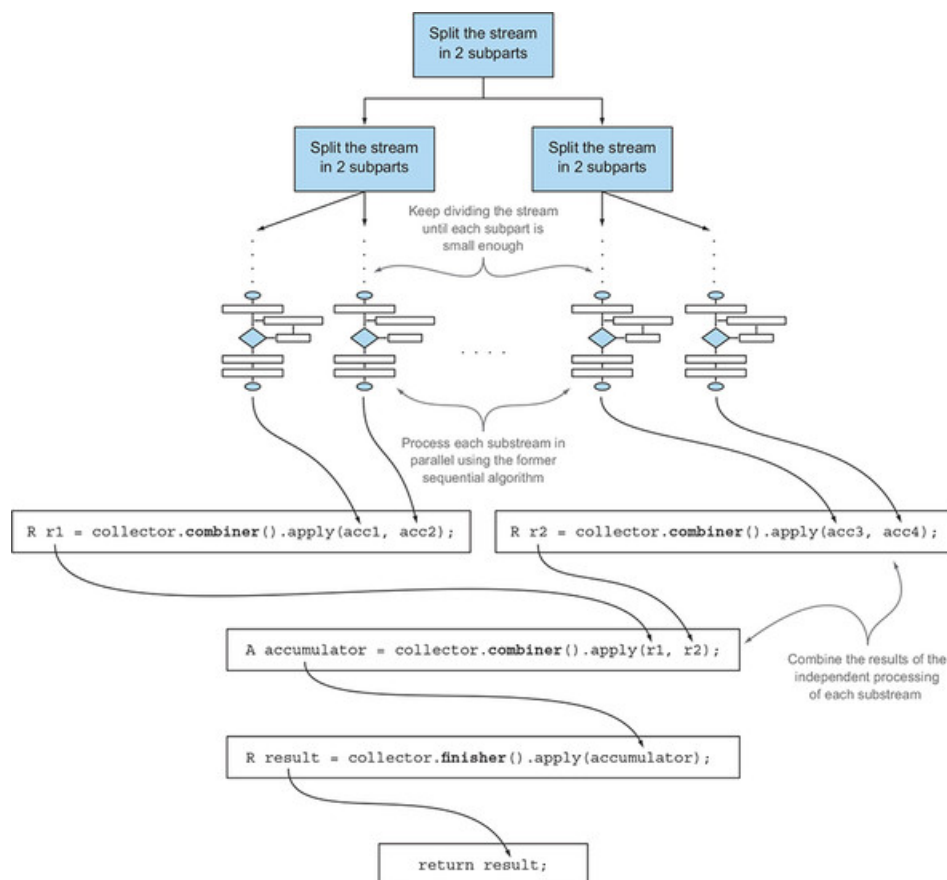


lower than sequential computing when subtasks are too small, and it's pointless to have more tasks than you have processing cores).

- At this point, all *substreams* can be processed in parallel, each of them using the sequential reduction algorithm shown in [figure 6.7](#).
- Finally, all the partial results are combined pairwise using the function returned by the combiner method of the collector. This is done by

combining results corresponding to substreams associated with each split of the original stream.

Figure 6.8. Parallelizing the reduction process using the combiner method



The characteristics method

The last method, `characteristics`, returns an immutable set of `Characteristics`, defining the behavior of the collector—in particular providing hints about whether the stream can be reduced in parallel and which optimizations are valid when doing so. `Characteristics` is an enumeration containing three items:

- **UNORDERED**— The result of the reduction isn't affected by the order in which the items in the stream are traversed and accumulated.
- **CONCURRENT**— The accumulator function can be called concurrently from multiple threads, and then this collector can perform a parallel reduction of the stream. If the collector isn't also flagged as **UNORDERED**, it can perform a parallel reduction only when it's applied to an unordered data source.

Other formats available



Audiobook



× indicates the function returned by the function, and its application can be omitted. In this case, the object is directly used as the final result of the function. This also implies that it's safe to do an accumulator A to the result R.

The `ToListCollector` developed so far is `IDENTITY_FINISH`, because the `List` used to accumulate the elements in the stream is already the expected final result and doesn't need any further transformation, but it isn't `UNORDERED` because if you apply it to an ordered stream you want this ordering to be preserved in the resulting `List`. Finally, it's `CONCURRENT`, but following what we just said, the stream will be processed in parallel only if its underlying data source is unordered.

6.5.2. Putting them all together

The five methods analyzed in the preceding subsection are everything you need to develop your own `ToListCollector` so you can implement it by putting all of them together, as the next listing shows.

Listing 6.5. The `ToListCollector`

```
import java.util.*;
import java.util.function.*;
import java.util.stream.Collector;
import static java.util.stream.Collector.Characteristics.*;
public class ToListCollector<T> implements Collector<T, List<T>, List<T>>
    @Override
    public Supplier<List<T>> supplier() {
        return ArrayList::new;
    }
    @Override
    public BiConsumer<List<T>, T> accumulator() {
        return List::add;
    }
    @Override
    public Function<List<T>, List<T>> finisher() {
        return Function.identity();
    }
    @Override
    public BinaryOperator<List<T>> combiner() {
        return (list1, list2) -> {
            list1.addAll(list2);
            return list1;
        };
    }
    @Override
    public Set<Characteristics> characteristics() {
        return Collections.unmodifiableSet(EnumSet.of(
            IDENTITY_FINISH, CONCURRENT));
    }
```

Other formats available



Audiobook



- **1** Creates the collection operation starting point
- **2** Accumulates the traversed item, modifying the accumulator in place
- **3** Identifies function

- **4 Modifies the first accumulator, combining it with the content of the second one**
- **5 Returns the modified first accumulator**
- **6 Flags the collector as `IDENTITY_FINISH` and `CONCURRENT`**

Note that this implementation isn't identical to the one returned by the `Collectors.toList()` method, but it differs only in some minor optimizations. These optimizations are mostly related to the fact that the collector provided by the Java API uses the `Collections.emptyList()` singleton when it has to return an empty list. This means that it could be safely used in place of the original Java as an example to gather a list of all the Dishes of a menu stream:

```
List<Dish> dishes = menuStream.collect(new ToListCollector<Dish>());
```

The remaining difference from this and the standard formulation

```
List<Dish> dishes = menuStream.collect(toList());
```

is that `toList` is a factory, whereas you have to use `new` to instantiate your `ToListCollector`.

Performing a custom collect without creating a Collector implementation

In the case of an `IDENTITY_FINISH` collection operation, there's a further possibility of obtaining the same result without developing a completely new implementation of the `Collector` interface. Streams has an overloaded `collect` method accepting the three other functions—supplier, accumulator, and combiner—having exactly the same semantics as the ones returned by the corresponding methods of the `Collector` interface. For instance, it's possible to collect in a `List` all the items in a stream of dishes, as follows:

```
List<Dish> dishes = menuStream.collect(
    ArrayList::new,           1
    List::add,                2
    List::addAll);           3
```

Other formats available

 Audiobook 

We believe that this second form, even if more compact and concise than the former one, is rather less readable. Also, developing an implementation of your custom collector in a proper class promotes its reuse and

helps avoid code duplication. It's also worth noting that you're not allowed to pass any `Characteristics` to this second `collect` method, so it always behaves as an `IDENTITY_FINISH` and `CONCURRENT` but not `UNORDERED` collector.

In the next section, you'll take your new knowledge of implementing collectors to the next level. You'll develop your own custom collector for a more complex but hopefully more specific and compelling use case.

6.6. Developing your own collector for better performance

In [section 6.4](#), where we discussed partitioning, you created a collector using one of the many convenient factory methods provided by the `Collectors` class, which divides the first n natural numbers into primes and nonprimes, as shown in the following listing.

Listing 6.6. Partitioning the first n natural numbers into primes and nonprimes

```
public Map<Boolean, List<Integer>> partitionPrimes(int n) {
    return IntStream.rangeClosed(2, n).boxed()
        .collect(partitioningBy(candidate -> isPrime(candidate)))
}
```

There you achieved an improvement over the original `isPrime` method by limiting the number of divisors to be tested against the candidate prime to those not bigger than the candidate's square root:

```
public boolean isPrime(int candidate) {
    int candidateRoot = (int) Math.sqrt((double) candidate);
    return IntStream.rangeClosed(2, candidateRoot)
        .noneMatch(i -> candidate % i == 0);
}
```

Is there a way to obtain even better performances? The answer is yes, but for this you'll have to develop a custom collector.

6.6.1. Divide only by prime numbers

One possible optimization is to test only if the candidate number is divisible to only the prime numbers found before

Other formats available



less to test it against a divisor that's not it-
to only the prime numbers found before
em with the predefined collectors you've
have to develop a custom one, is that dur-

ing the collecting process you don't have access to the partial result. This means that when testing whether a given candidate number is prime or not, you don't have access to the list of the other prime numbers found so far.

Suppose you had this list; you could pass it to the `isPrime` method and rewrite it as follows:

```
public static boolean isPrime(List<Integer> primes, int candidate) {  
    return primes.stream().noneMatch(i -> candidate % i == 0);  
}
```

Also, you should implement the same optimization you used before and test only with primes smaller than the square root of the candidate number. You need a way to stop testing whether the candidate is divisible by a prime as soon as the next prime is greater than the candidate's root. You can easily do this by using the Stream's `takeWhile` method:

```
public static boolean isPrime(List<Integer> primes, int candidate){  
    int candidateRoot = (int) Math.sqrt((double) candidate);  
    return primes.stream()  
        .takeWhile(i -> i <= candidateRoot)  
        .noneMatch(i -> candidate % i == 0);  
}
```

Quiz 6.3: Simulating `takeWhile` in Java 8

The `takeWhile` method was introduced in Java 9, so unfortunately you cannot use this solution if you are still using Java 8. How could you work around this limitation and achieve something similar in Java 8?

Answer:

You could implement your own `takeWhile` method, which, given a sorted list and a predicate, returns the longest prefix of this list whose elements satisfy the predicate:

```
public static <A> List<A> takeWhile(List<A> list, Predicate<A> p) {  
    int i = 0;  
    for (A item : list) {  
        if (!p.test(item)) {  
            return list.subList(0, i);  
        }  
        i++;  
    }  
}
```

Other formats available



Audiobook



- 1 Checks if the current item in the list satisfies the Predicate
- 2 If it doesn't, returns the sublist prefix until the item before the tested one

- **3 All the items in the list satisfy the Predicate, so returns the list itself**

Using this method, you can rewrite the `isPrime` method and once again testing only the candidate prime against only the primes that aren't greater than its square root:

```
public static boolean isPrime(List<Integer> primes, int candidate){
    int candidateRoot = (int) Math.sqrt((double) candidate);
    return takeWhile(primes, i -> i <= candidateRoot)
        .stream()
        .noneMatch(p -> candidate % p == 0);
}
```

Note that, unlike the one provided by the Streams API, this implementation of `takeWhile` is eager. When possible, always prefer the Java 9 Stream's lazy version of `takeWhile` so it can be merged with the `noneMatch` operation.

With this new `isPrime` method in hand, you're now ready to implement your own custom collector. First, you need to declare a new class that implements the `Collector` interface. Then, you need to develop the five methods required by the `Collector` interface.

Step 1: Defining the Collector class signature

Let's start with the class signature, remembering that the `Collector` interface is defined as

```
public interface Collector<T, A, R>
```

where `T`, `A`, and `R` are respectively the type of the elements in the stream, the type of the object used to accumulate partial results, and the type of the final result of the `collect` operation. In this case, you want to collect streams of `Integers` while both the accumulator and the result types are `Map<Boolean, List<Integer>>` (the same `Map` you obtained as a result of the former partitioning operation in [listing 6.6](#)), having as keys `true` and `false` respectively the Lists of prime and nonprime

Other formats available



Audiobook



```
public class PrimeNumbersCollector
```

```
implements Collector<Integer,
```

```
    Map<Boolean, List<Integer>>,
```

```
    Map<Boolean, List<Integer>>>
```

1

2

3

- 1 The type of the elements in the stream
- 2 The type of the accumulator
- 3 The type of the result of the collect operation

Step 2: Implementing the reduction process

Next, you need to implement the five methods declared in the `Collector` interface. The `supplier` method has to return a function that when invoked creates the accumulator:

```
public Supplier<Map<Boolean, List<Integer>>> supplier() {
    return () -> new HashMap<Boolean, List<Integer>>() {{
        put(true, new ArrayList<Integer>());
        put(false, new ArrayList<Integer>());
    }};
}
```

Here you're not only creating the `Map` that you'll use as the accumulator, but you're also initializing it with two empty lists under the `true` and `false` keys. This is where you'll add respectively the prime and non-prime numbers during the collection process. The most important method of your collector is the `accumulator` method, because it contains the logic defining how the elements of the stream have to be collected. In this case, it's also the key to implementing the optimization we described previously. At any given iteration you can now access the partial result of the collection process, which is the accumulator containing the prime numbers found so far:

```
public BiConsumer<Map<Boolean, List<Integer>>, Integer> accumulator() {
    return (Map<Boolean, List<Integer>> acc, Integer candidate) -> {
        acc.get( isPrime(acc.get(true), candidate) )
        .add(candidate);
    };
}
```

- 1 Gets the list of prime or nonprime numbers depending on the result of `isPrime`
- 2 Adds the candidate to the appropriate list

In this method, you invoke the `isPrime` method, passing to it (together

Other formats available



✗ want to test whether it's prime or not) the so far. (These are the values indexed by `g Map`.) The result of this invocation is

then used as the key to get the list of either the prime or nonprime numbers, so you can add the new candidate to the right list.

Step 3: Making the collector work in parallel (if possible)

The next method has to combine two partial accumulators in the case of a parallel collection process, so in this case it has to merge the two Maps by adding all the numbers in the prime and nonprime lists of the second Map to the corresponding lists in the first Map:

```
public BinaryOperator<Map<Boolean, List<Integer>>> combiner() {
    return (Map<Boolean, List<Integer>> map1,
            Map<Boolean, List<Integer>> map2) -> {
        map1.get(true).addAll(map2.get(true));
        map1.get(false).addAll(map2.get(false));
        return map1;
    };
}
```

Note that in reality this collector can't be used in parallel, because the algorithm is inherently sequential. This means the combiner method won't ever be invoked, and you could leave its implementation empty (or better, throw an `UnsupportedOperationException`). We decided to implement it anyway only for completeness.

Step 4: The finisher method and the collector's characteristic method

The implementation of the last two methods is quite straightforward. As we said, the accumulator coincides with the collector's result so it won't need any further transformation, and the finisher method returns the identity function:

```
public Function<Map<Boolean, List<Integer>>,
               Map<Boolean, List<Integer>>> finisher() {
    return Function.identity();
}
```

As for the characteristic method, we already said that it's neither `CONCURRENT` nor `UNORDERED` but is `IDENTITY_FINISH`:

```
public Set<Characteristics> characteristics() {
    return Collections.unmodifiableSet(EnumSet.of(IDENTITY_FINISH));
}
```

Other formats available



Final implementation of

Collector

```
public class PrimeNumbersCollector
    implements Collector<Integer,
                       Map<Boolean, List<Integer>>,
```

```

        Map<Boolean, List<Integer>>> {
    @Override
    public Supplier<Map<Boolean, List<Integer>>> supplier() {
        return () -> new HashMap<Boolean, List<Integer>>>() {{
            put(true, new ArrayList<Integer>());
            put(false, new ArrayList<Integer>());
        }};
    }
    @Override
    public BiConsumer<Map<Boolean, List<Integer>>, Integer> accumulator(
        return (Map<Boolean, List<Integer>> acc, Integer candidate) -> {
            acc.get( isPrime( acc.get(true),
                candidate) )
                .add(candidate);
        };
    }
    @Override
    public BinaryOperator<Map<Boolean, List<Integer>>> combiner() {
        return (Map<Boolean, List<Integer>> map1,
            Map<Boolean, List<Integer>> map2) -> {
            map1.get(true).addAll(map2.get(true));
            map1.get(false).addAll(map2.get(false));
            return map1;
        };
    }
    @Override
    public Function<Map<Boolean, List<Integer>>,
        Map<Boolean, List<Integer>>> finisher() {
        return Function.identity();
    }
    @Override
    public Set<Characteristics> characteristics() {
        return Collections.unmodifiableSet(EnumSet.of(IDENTITY_FINISH));
    }
}

```

- **1 Starts the collection process with a Map containing two empty Lists**
- **2 Passes to the isPrime method the list of already found primes**
- **3 Gets from the Map the list of prime or nonprime numbers, according to what the isPrime method returned, and adds to it the current candidate**
- **4 Merges the second Map into the first one**
- **5 No transformation necessary at the end of the collection process,**

Other formats available



✕ Identity function

IDENTITY_FINISH but neither UNORDERED nor

parallelism. It relies on the fact that prime numbers are

discovered in sequence.

You can now use this new custom collector in place of the former one created with the partitioningBy factory method in [section 6.4](#) and obtain

exactly the same result:

```
public Map<Boolean, List<Integer>>
    partitionPrimesWithCustomCollector(int n) {
    return IntStream.rangeClosed(2, n).boxed()
        .collect(new PrimeNumbersCollector());
}
```

6.6.2. Comparing collectors' performances

The collector created with the `partitioningBy` factory method and the custom one you just developed are functionally identical, but did you achieve your goal of improving the performance of the `partitioningBy` collector with your custom one? Let's write a quick harness to check this:

```
public class CollectorHarness {
    public static void main(String[] args) {
        long fastest = Long.MAX_VALUE;
        for (int i = 0; i < 10; i++) {
            long start = System.nanoTime();
            partitionPrimes(1_000_000);
            long duration = (System.nanoTime() - start) / 1_000_000;
            if (duration < fastest) fastest = duration;
        }
        System.out.println(
            "Fastest execution done in " + fastest + " msecs");
    }
}
```

- **1 Runs the test 10 times**
- **2 Partitions the first million natural numbers into primes and nonprimes**
- **3 The duration in milliseconds**
- **4 Checks if this execution is the fastest one**

Note that a more scientific benchmarking approach would be to use a framework such as Java Microbenchmark Harness (JMH), but we didn't want to add the complexity of using such a framework here and, for this use case, the results provided by this small benchmarking class are accurate enough. This class partitions the first million natural numbers into primes and nonprimes, invoking the method using the collector created

Other formats available



✕ry method 10 times and registering the
an Intel i5 2.4 GHz, it prints the following

```
Fastest execution done in 4716 msecs
```

Now replace `partitionPrimes` with `partitionPrimesWithCustomCollector` in the harness, in order to test the performances of the custom collector you developed. Now the program prints

```
Fastest execution done in 3201 msecs
```

Not bad! This means you didn't waste your time developing this custom collector for two reasons: First, you learned how to implement your own collector when you need it. And second, you achieved a performance improvement of around 32%.

Finally, it's important to note that, as you did for the `ToListCollector` in [listing 6.5](#), it's possible to obtain the same result by passing the three functions implementing the core logic of `PrimeNumbersCollector` to the overloaded version of the `collect` method, taking them as arguments:

```
public Map<Boolean, List<Integer>> partitionPrimesWithCustomCollector
    (int n) {
    IntStream.rangeClosed(2, n).boxed()
        .collect(
            () -> new HashMap<Boolean, List<Integer>>() {{
                put(true, new ArrayList<Integer>());
                put(false, new ArrayList<Integer>());
            }},
            (acc, candidate) -> {
                acc.get( isPrime(acc.get(true), candidate) )
                    .add(candidate);
            },
            (map1, map2) -> {
                map1.get(true).addAll(map2.get(true));
                map1.get(false).addAll(map2.get(false));
            }
        ));
}
```

- **1 Supplier**
- **2 Accumulator**
- **3 Combiner**

As you can see, in this way you can avoid creating a completely new class that implements the `Collector` interface; the resulting code is more compact even if it's also probably less readable and certainly less

Other formats available

 Audiobook 

- `collect` is a terminal operation that takes as argument various recipes (called collectors) for accumulating the elements of a stream into a summary result.

- Predefined collectors include reducing and summarizing stream elements into a single value, such as calculating the minimum, maximum, or average. Those collectors are summarized in [table 6.1](#).
- Predefined collectors let you group elements of a stream with `groupingBy` and partition elements of a stream with `partitioningBy`.
- Collectors compose effectively to create multilevel groupings, partitions, and reductions.
- You can develop your own collectors by implementing the methods defined in the `Collector` interface.

Other formats available



Audiobook

